



UNIVERSITÀ  
degli STUDI  
di CATANIA

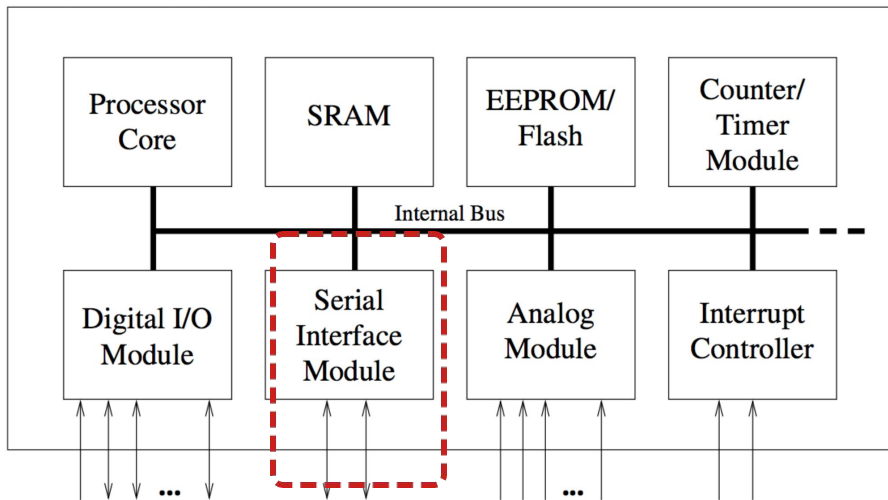
IoT Systems and Technologies

# Introduction to Microcontrollers Communication Interfaces

davide.patti@dieei.unict.it

# Communication Protocols

Microcontroller



## Beyond Simple Voltage

- Pins transmit/receive **complex messages**, not just static electrical levels.
- Enables **sensor commands** and data requests from actuators.
- Facilitates **multi-device coordination** on a shared physical line.

**IMPORTANT:** While technically transmitting 1s and 0s, these follow a **Communication Protocol** to encode and deliver structured information.

# Interface Classification



## Serial vs. Parallel

Data sequence and line count



## Sync vs. Async

Timing and clock coordination



## Bus vs. Point-to-Point

Shared medium or dedicated link



## Full vs. Half Duplex

Bi-directional data flow control



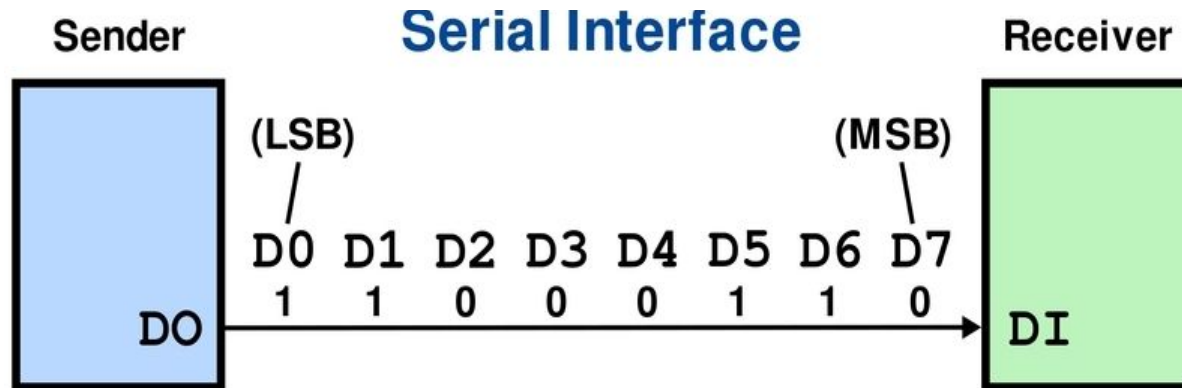
## Master-Slave vs. Peer-to-Peer

Hierarchy and control architecture

**These criteria define the physical and logical constraints of hardware communication.**

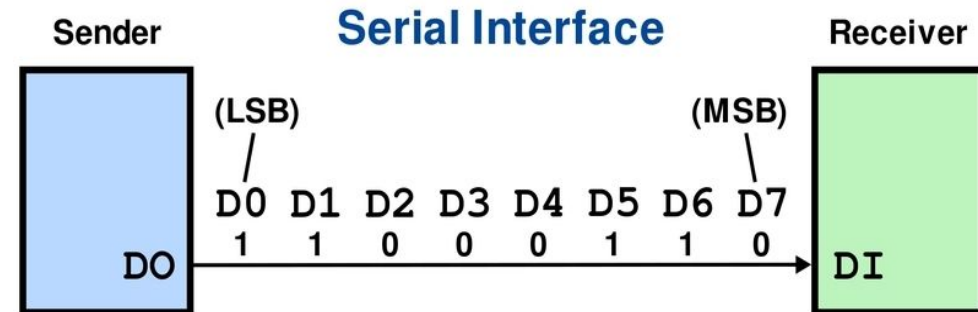
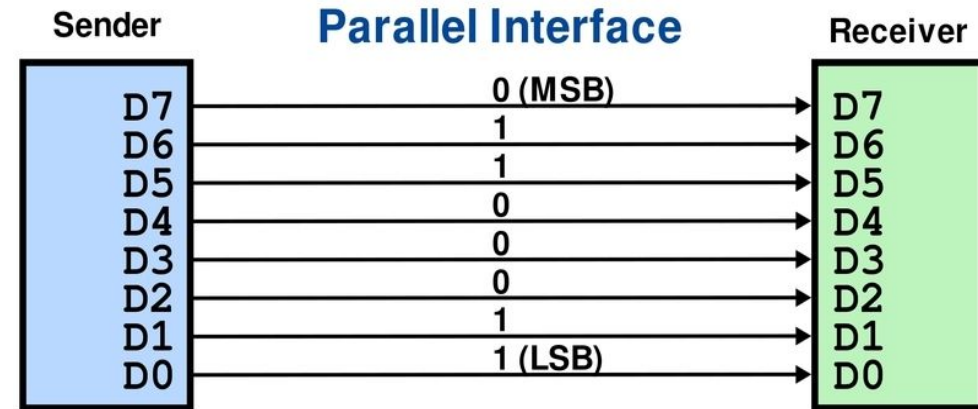
# Serial Interface

- Sends data sequentially, one bit at a time
- Requires only one data line
  - **(Pros)** Communication is resource efficient
  - **(Cons)** Throughput is lower (for a fixed transmission rate)



# Parallel Interface

- Several data lines to transfer more than one bit at a time
- The number of bits that are transferred in parallel varies
- **(Pros)** More bits per unit of time
- **(Cons)** Lines will be allocated to the interface even when they would be not necessary (pins are limited!)



# Synchronous Interface

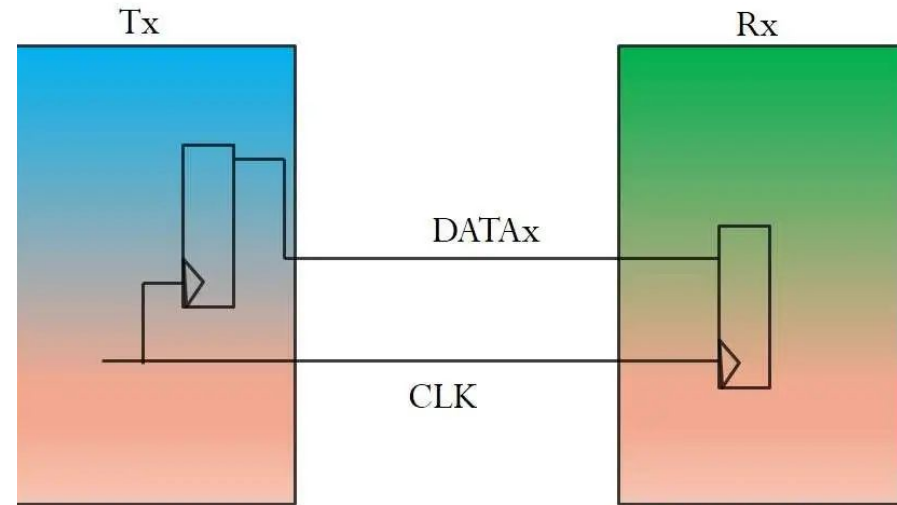
## Linked Clocks

The receive clock is linked to the send clock via an additional shared clock line.

## Key Advantages (Pros)

- Receiver doesn't need to generate its own clock
- Lower susceptibility to synchronization errors

Timing Diagram & Connection Architecture



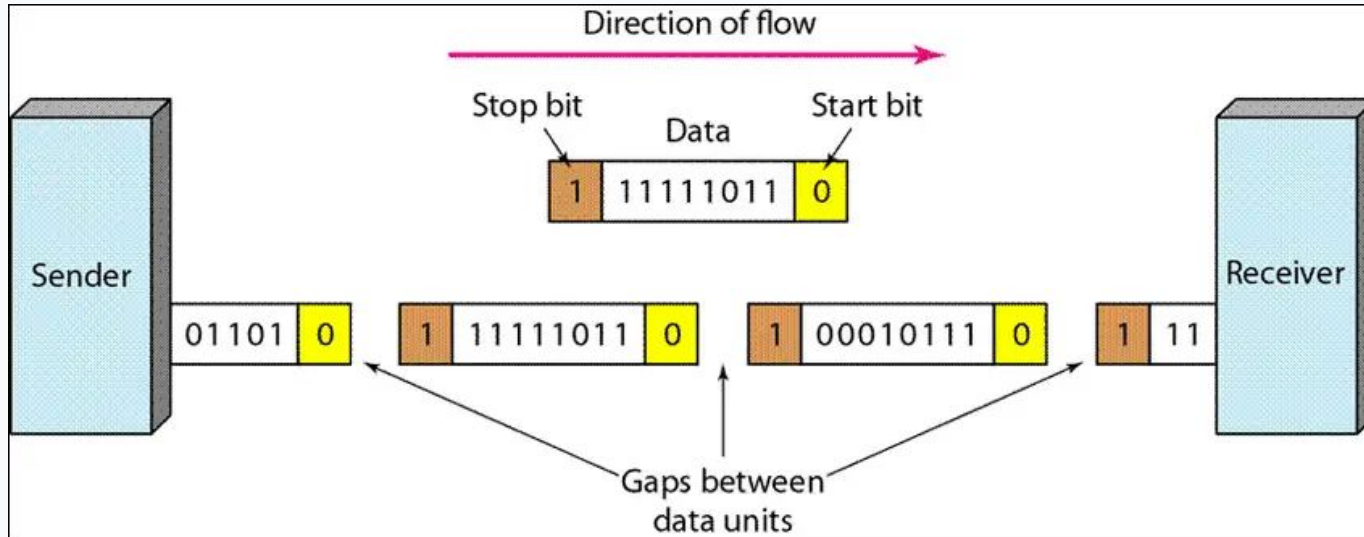
# Asynchronous Interface

## Independent Clocks

Send and receive clocks are not linked, unlike synchronous systems.

## Key Mechanisms

- **Oversampling:** Receiver uses a higher frequency to detect timing.
- **Start/Stop Bits:** Special framing bits identify the data block boundaries.



# Bus Connection

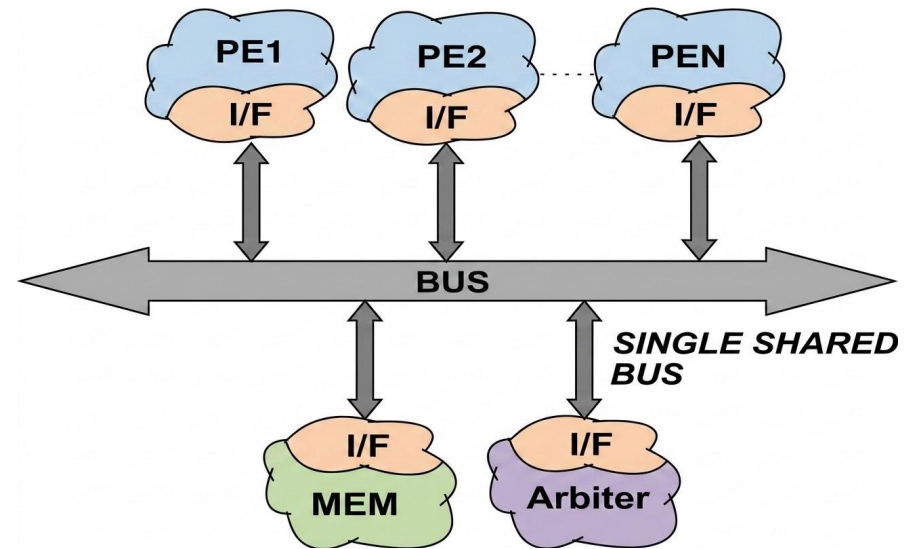
## Multi-Device Support

More than two devices can be connected to the shared communication medium.

## Addressing Mechanism

Since all devices share the bus, a form of addressing is required to select the intended recipient.

Shared Bus Architecture Diagram





# Point-to-Point Connection

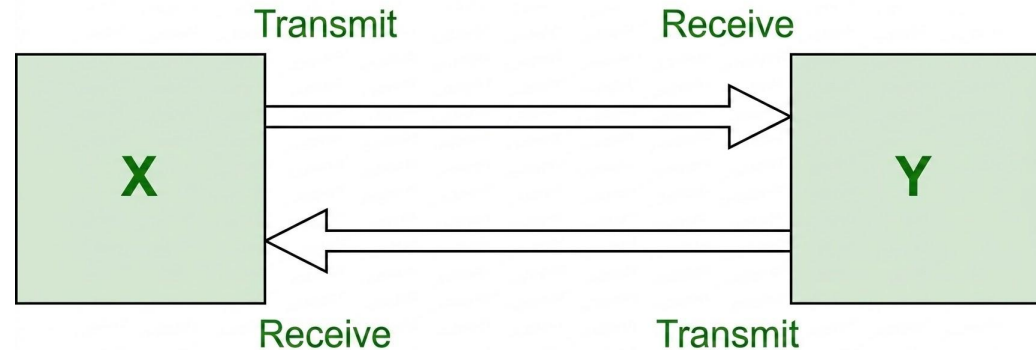
## Direct Communication

Specifically designed for communication between just two dedicated devices.

## Implicit Addressing

No addressing required; it is inherently clear who the receiver of a message will be.

Transmission Architecture



# Bi-directional Communication

In most cases, data communication between a controller and its peripherals is bi-directional: both devices will transmit data.

**Can both devices transmit at the same time?**

## Full Duplex

Simultaneous bi-directional transmission.

- Both sides transmit at once
- Requires dedicated paths (wires)
- Maximum throughput

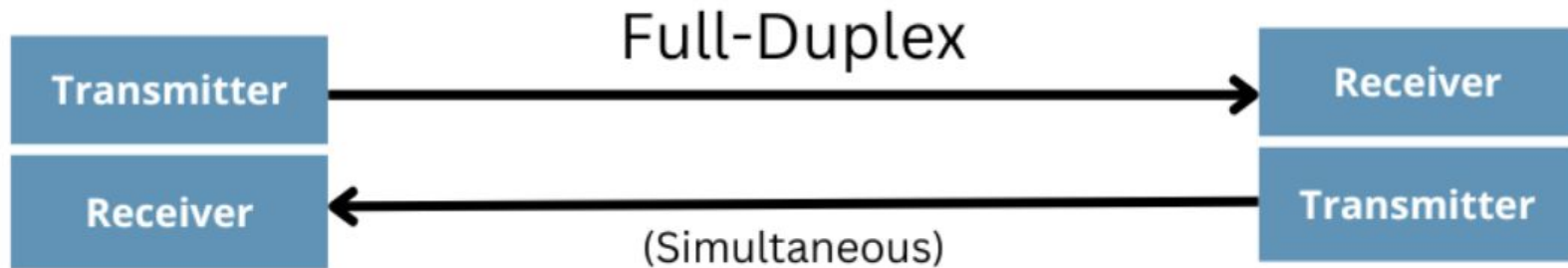
## Half Duplex

Alternating bi-directional transmission.

- Only one node transmits at a time
- Saves on wiring costs
- Requires access negotiation

# Full-duplex Connection

- Both sides can transmit **at the same time**
- This requires at least two wires
- Allows a maximum overall throughput



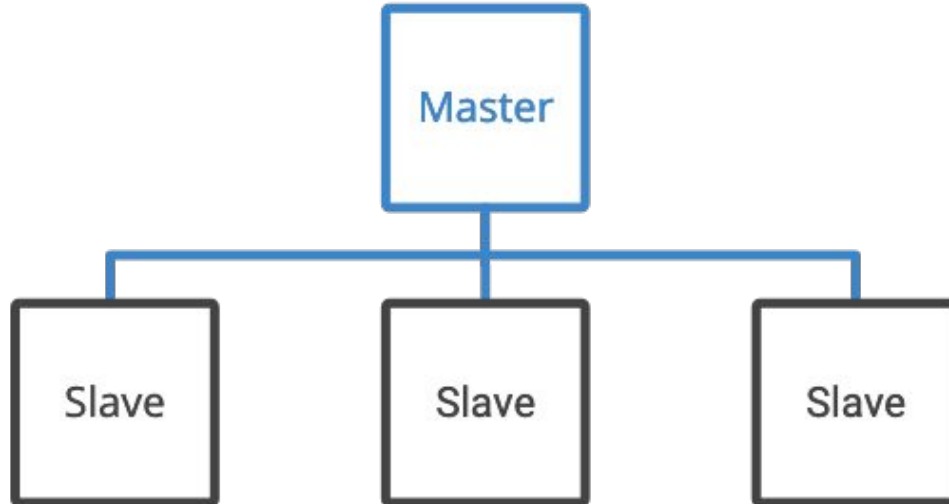
# Half-duplex Connection

- Only one node transmits at any time
- **(Pros)** Saves on wires
- **(Cons)** Less overall throughput and the necessity of negotiating access to the single wire



# Master-Slave Systems

- Only the master can initiate a transmission
- Slaves must wait for the master to allow them access to the communication medium



# Characterization at Physical Layer

## Communication Interface Types

### Single-ended



Uses one wire to carry the signal and shares a common ground reference.

- Simple and cost-effective
- Sensitive to noise and interference
- Limited distance for high speeds

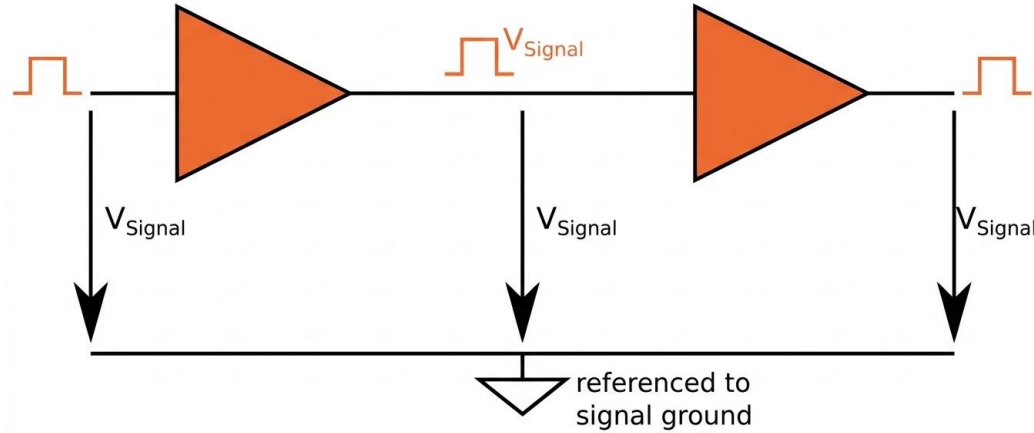
### Differential



Uses a pair of wires to carry equal and opposite signals.

- High immunity to common-mode noise
- Reliable over longer distances
- Standard for high-speed protocols

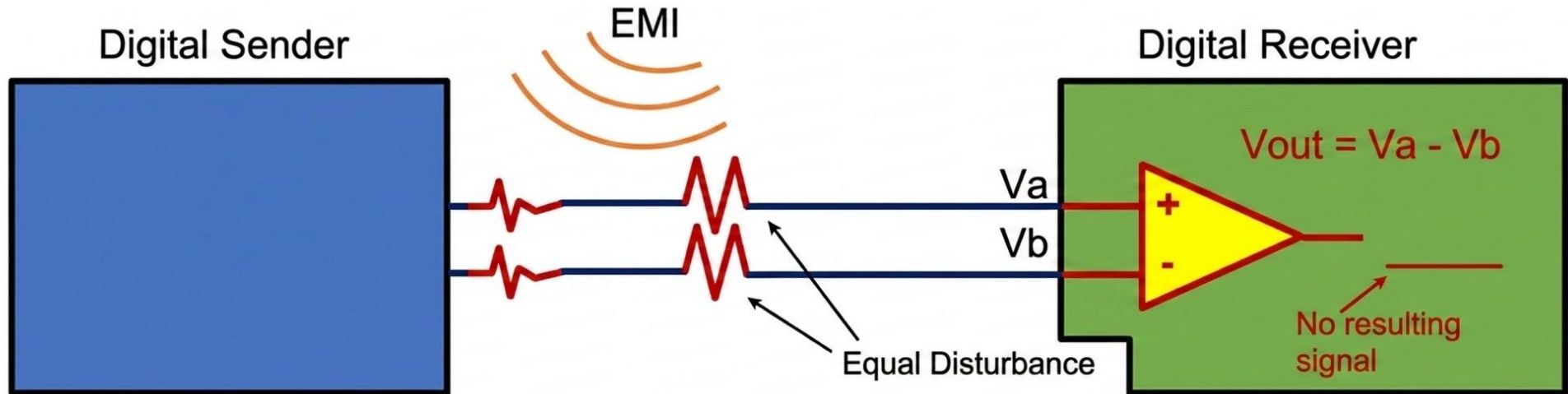
# Single-ended



- Voltage levels of the wires with respect to ground
- Sender and receiver share the same ground
- Problem if sender and receiver are far away, since interferences can be act differently on the signal

# Differential Interface

- Use two wires to transmit a signal
- The voltage difference between the two wires carries the signal





# Differential Interface

Signals are transmitted using **two wires** where the **voltage difference** between them carries the information.

## Cons



Requires **two wires** instead of the single line used in single-ended connections.

## Pros



High **Noise Immunity**: interferences affect both wires equally, preserving the voltage difference.



Supports **longer transmission lines** compared to single-ended interfaces.

# Communication Protocols

- UART
- SPI
- IIC

# SCI (UART) Overview

The **Serial Communication Interface (SCI)** provides an asynchronous communication interface known as **Universal Asynchronous Receiver Transmitter (UART)**.

## Wiring



Utilizes **two wires**:

- **TXD**: Transmit line
- **RXD**: Receive line

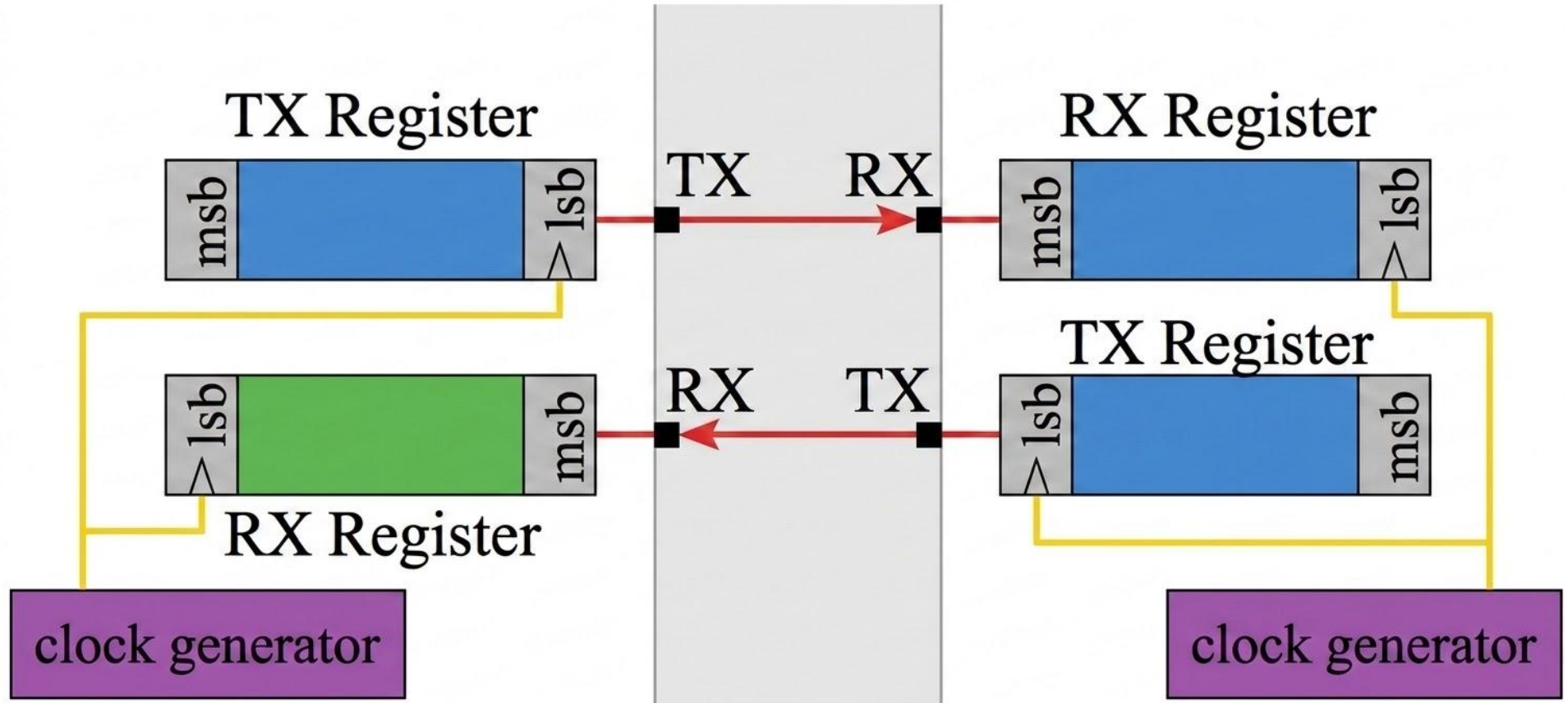
## Operation



Supports flexible communication modes:

- **Full-duplex**
- **Half-duplex**

# Basic Structure of a UART Module



# UART Configurable Parameters

## Data Structure



### Number of Data Bits

Defines the payload length in each frame, typically ranging from 5 to 9 bits.

## Error Checking



### Parity Bit

Optional bit for error detection: None, Even, or Odd parity configurations.

## Synchronization



### Stop Bits

Signals the end of a character transmission, commonly 1, 1.5, or 2 bits.

## Transmission Speed



### Baud Rate

Determines the serial data rate (bits per second). Must match on both sides.

# UART: Number of Data Bits

## Payload Configuration

### Flexible Data Range

The number of data bits in a UART frame is configurable within a defined range, allowing for optimization based on the specific application needs.



**Typical range: 5 to 9 data bits**

# UART: Parity Bit Configuration

## Error Detection Logic

### Configurability

The parity bit is an optional tool for error detection. You must first decide whether to include it, and then specify the parity type (Odd or Even).

| 7 bits of data | (count of 1-bits) | 8 bits including parity |                  |
|----------------|-------------------|-------------------------|------------------|
|                |                   | even                    | odd              |
| 0000000        | 0                 | 0000000 <b>0</b>        | 0000000 <b>1</b> |
| 1010001        | 3                 | 1010001 <b>1</b>        | 1010001 <b>0</b> |
| 1101001        | 4                 | 1101001 <b>0</b>        | 1101001 <b>1</b> |
| 1111111        | 7                 | 1111111 <b>1</b>        | 1111111 <b>0</b> |

# UART: Stop Bit Configuration

## Frame Termination Logic

### Configurable Stop Bits

The stop bit serves to signal the end of a data byte. You can configure whether there should be one stop bit or two.

 **Note: Start bits are not configurable and are always one bit.**



# UART: Baud Rate & Speed

## Transmission Speed Definition

### Baud Rate vs. Bit Rate

Baud rate measures signal changes per second. In typical UART cases with two signal levels, 1 Baud = 1 bit/s.

## Standard Baud Rates

Typical ranges are between **9600** and **115200** baud.

### Common Values:

- 9600 bps (Standard)
- 19200 bps
- 38400 bps
- 115200 bps (High Speed)

## System Clock Dependency

Feasible baud rates depend directly on the **System Clock Frequency**.

Different clock speeds imply different sets of available baud rates. Precise timing is required to avoid sampling errors.



# UART: Nomenclature Guide

## Standard Data Format Representation

**D** { **E** | **O** | **N** } **S**

**D**

### Data Bits

Specifies the number of bits in the data payload (typically 5 to 9).

**E** | **O** | **N**

### Parity Type

- **E**: Even Parity
- **O**: Odd Parity
- **N**: No Parity

**S**

### Stop Bits

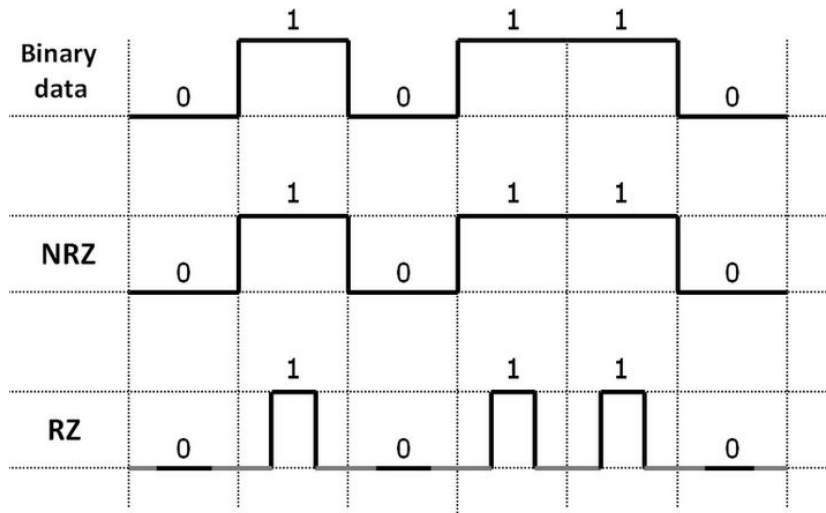
Indicates the number of stop bits used to terminate the frame (usually 1 or 2).

💡 **Example:** 8E1 = 8 Data bits, Even parity, 1 Stop bit.

# UART: Data Transmission Protocols

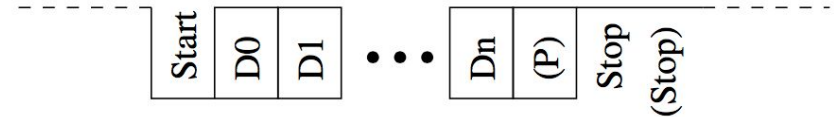
## NRZ Encoding

Non Return to Zero (NRZ) is the standard line code for asynchronous serial communication.



## General UART Packet Frame

A UART packet consists of a start bit, data bits, an optional parity bit, and stop bits.



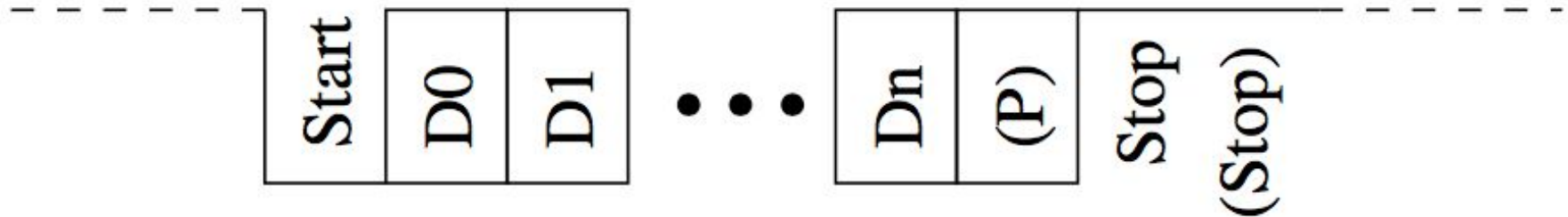
**Start Bit:** Always low (logical 0).

**Data (D0-Dn):** 5 to 9 bits of payload.

**Parity (P):** Optional error detection bit.

**Stop Bits:** One or two bits at high (logical 1).

# UART: Data Transmission Configuration



## Data Bits

The number of data bits is configurable (typically 5 to 9).

### **CRITICAL:**

Must be identical on both Sender and Receiver. Users are responsible for this sync.

## Parity Bit

An optional error detection bit that follows the data payload.

- Even Parity
- Odd Parity
- None

## Stop Bits

The frame is concluded by either one or two stop bits.

Used to signal the end of the packet and return the line to the idle state (high).

# Synchronization & Error Recognition

## Asynchronous Mode

Communication relies on independent clocks at both ends.

### KEY POINT:

Sender and receiver clocks are completely independent of each other.

## Baud Rate Role

Pre-selecting the baud rate informs the receiver about the expected bit rate.

### CHALLENGE:

The receiver knows the speed, but does not know exactly when a bit starts.

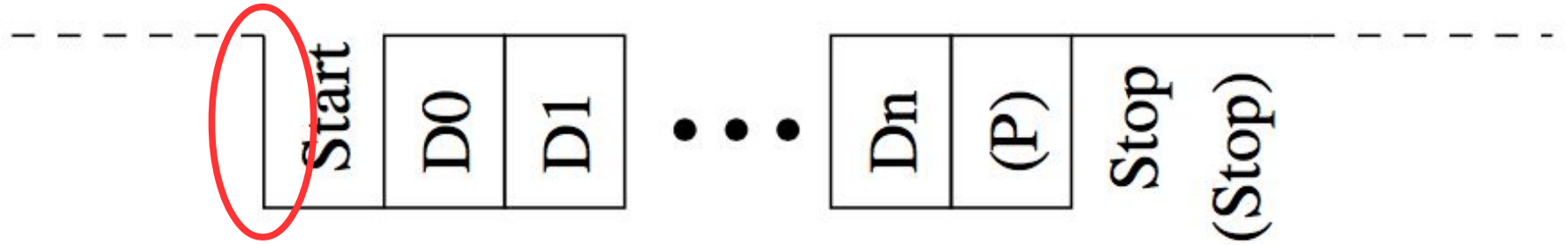
## Synchronization

The timing must be aligned to ensure accurate data capture.

### SOLUTION:

It needs to synchronize to the falling edge of the start bit.

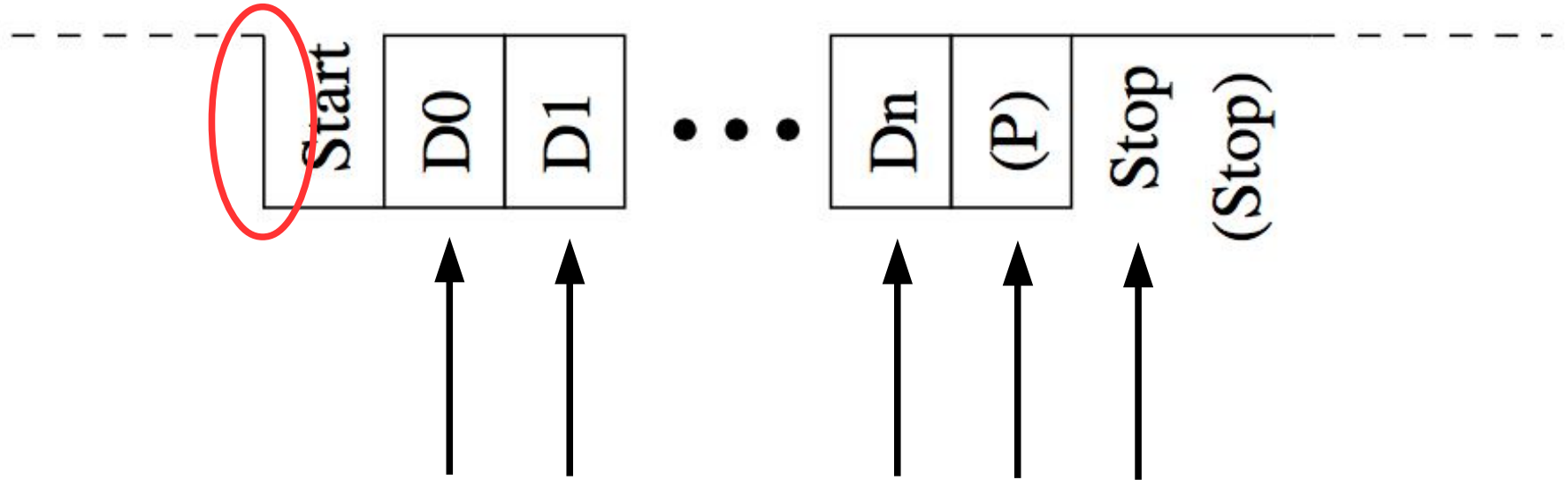
# Synchronization and Error Recognition (cnt'd)



Falling edge detected

If baud rate 9600, sample every  $1/9600$  sec

# Synchronization and Error Recognition (cnt'd)



# Synchronization and Error Recognition (cnt'd)

- Problem
  - Clock oscillators tend to have a non-zero drift
    - They deviate from their nominal frequency
  - Even if the receiver synchronizes to the sender clock at the start of the message, it might drift away during transmission



# Synchronization & Error Recognition

## The Clock Drift Problem

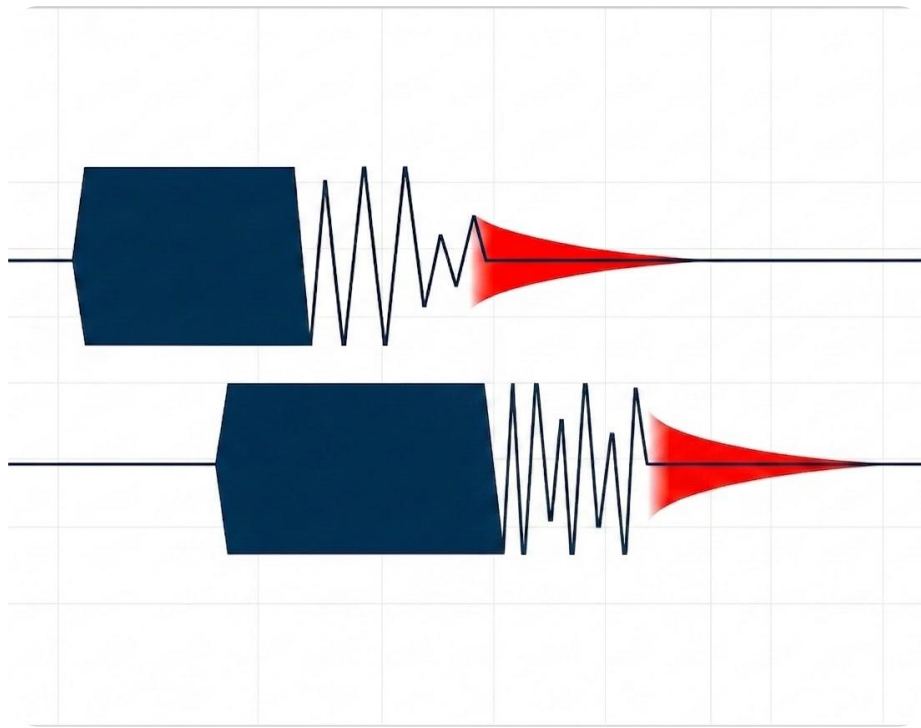
### The Problem

#### Clock Oscillator Drift

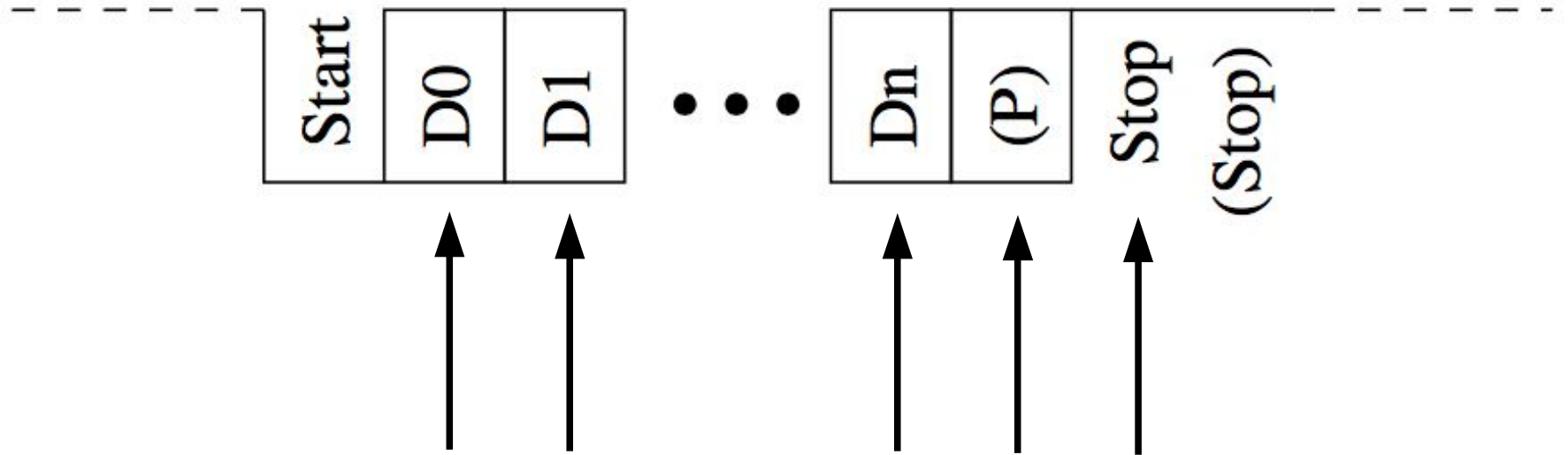
Oscillators have a non-zero drift, meaning they deviate from their nominal frequency over time.

#### Cumulative Error

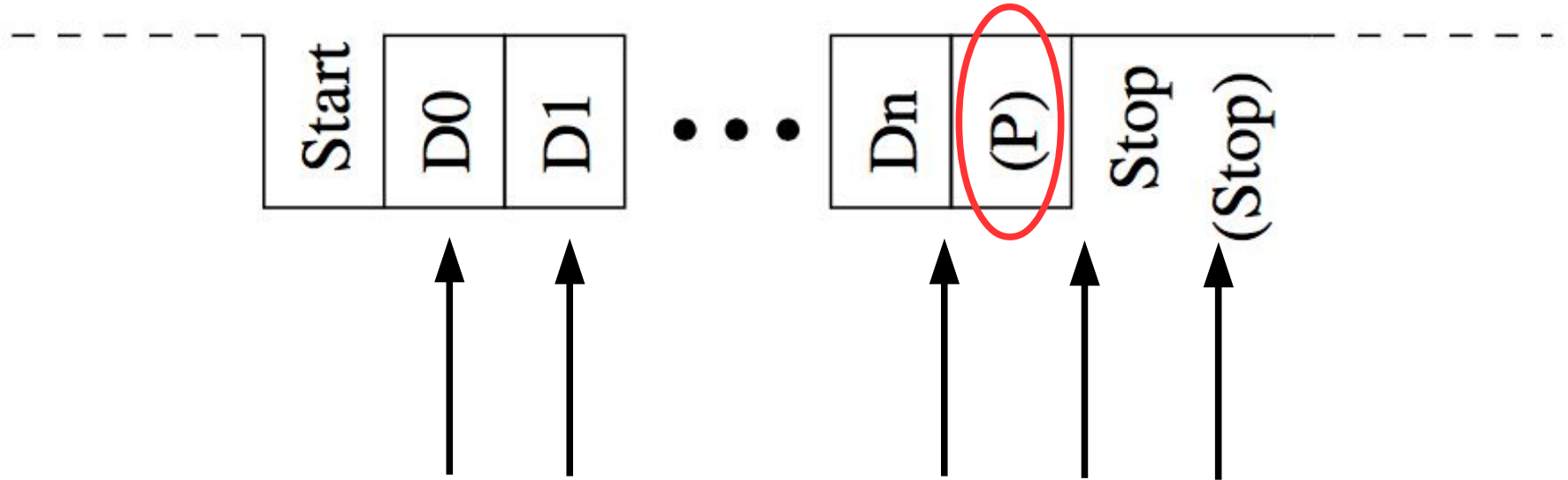
Even with initial synchronization, the receiver's clock might drift away during the transmission of a long message.



# Synchronization and Error Recognition (cnt'd)



# Synchronization and Error Recognition (cnt'd)



# Synchronization & Error Recognition



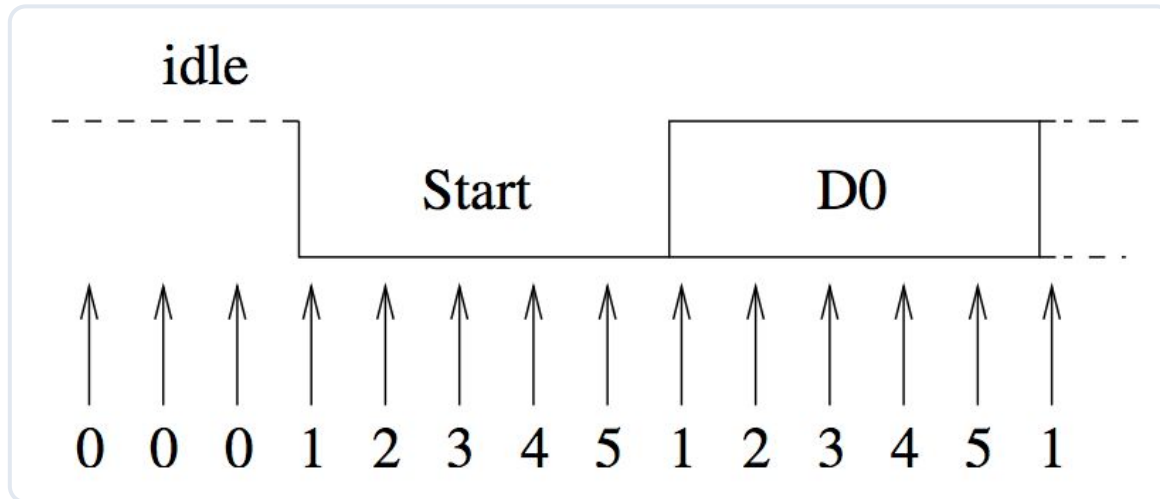
## Oversampling Mechanism

The receiver monitors the RXD line by sampling it multiple times within a single bit period to ensure timing accuracy.



## Standard Configuration

- **s samples per bit:** The rate of oversampling.
- **Typical value (s = 16):** Industry standard for robust error recognition.



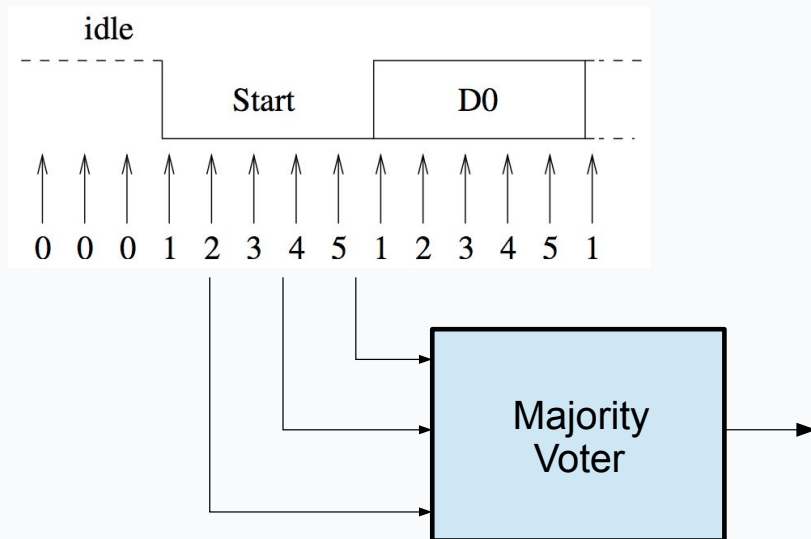
# Synchronization & Error Recognition (Cont'd)

## Start Bit Ideal Condition

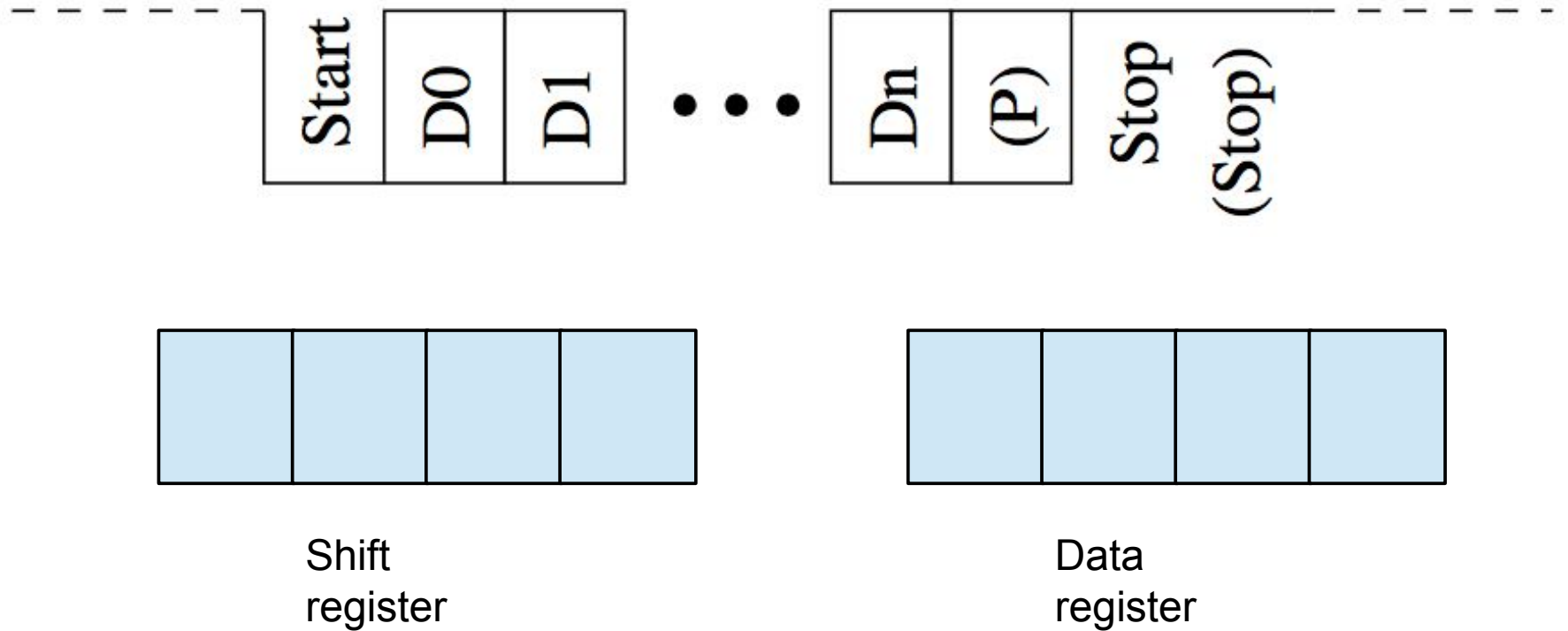


Ideally, all **s samples** of the start bit should be **zero**.

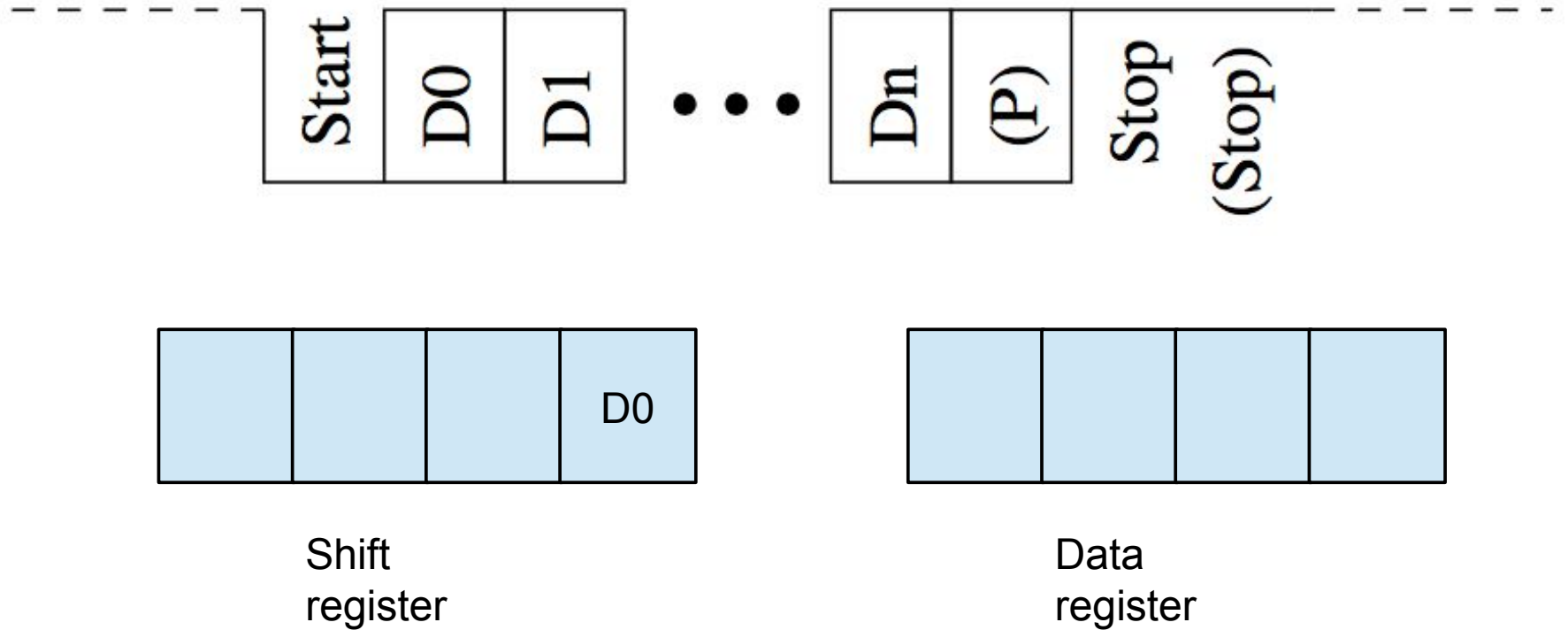
To be more resilient to noise, the UART only uses some of the samples to determine the value of the bit.



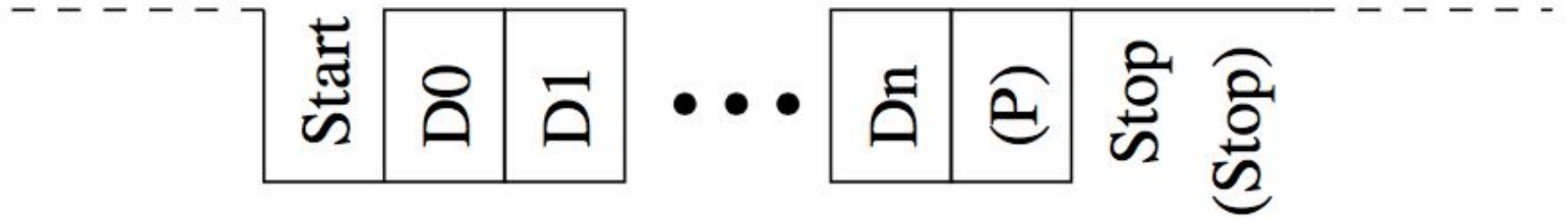
# Synchronization and Error Recognition (cnt'd)



# Synchronization and Error Recognition (cnt'd)



# Synchronization and Error Recognition (cnt'd)



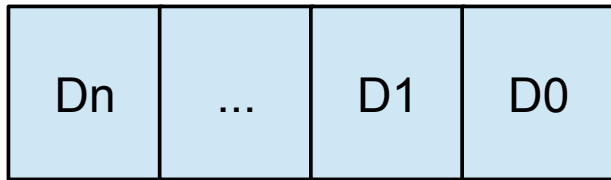
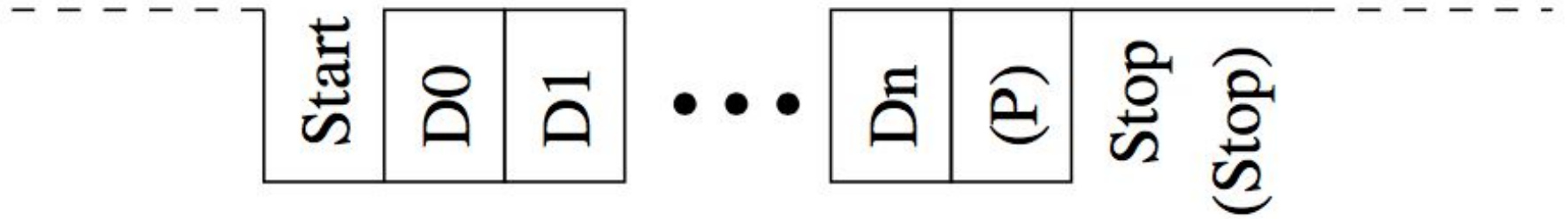
Shift  
register



Data  
register



# Synchronization and Error Recognition (cnt'd)

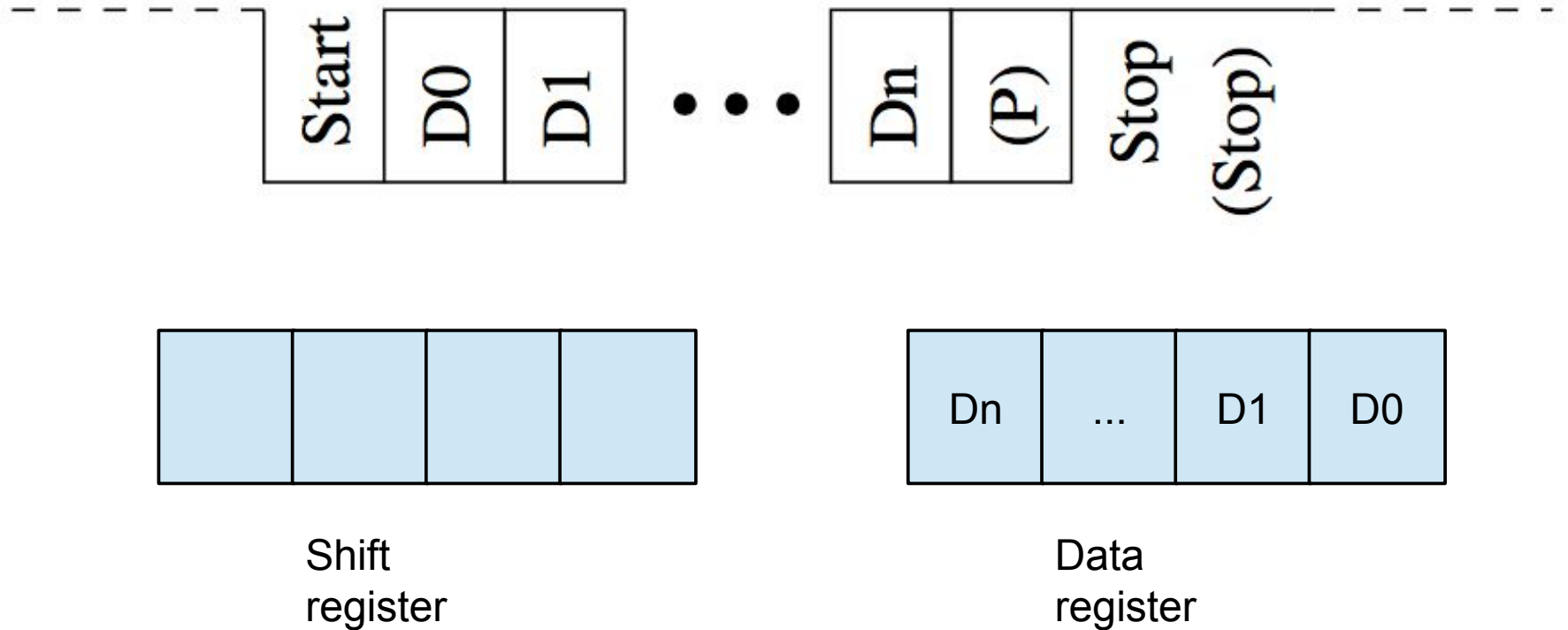


Shift  
register



Data  
register

# Synchronization and Error Recognition (cnt'd)



# Synchronization & Error Recognition



## Baud Rate Divergence

Baud rates between sender and receiver may diverge significantly, causing gradual synchronization loss during transmission.



## Stop Bit Detection

Drift is recognizable at the stop bit, where the UART expects a high level. Failure to read this level indicates an error.



## Frame Error Announcement

If the stop bit is not recognized due to baud rate mismatch or timing drift, a **frame error** is formally announced to the application layer.

# Synchronization & Error Recognition

## Data Overrun

A data overrun occurs when the processor does not read the received character from the UART buffer before the next one arrives.

- **Buffer Register:** The UART uses a buffer to provide the application more time to read incoming data.
- **Processing Delay:** A second transmission may start before the first has been fully processed.
- **Critical Failure:** A third transmission might begin before the first has even been read, leading to loss.



# Serial Peripheral Interface (SPI)



## Core Definition

SPI is a standard for Serial Peripheral Interface communication, commonly used in embedded systems.



## Topology

It functions as a synchronous point-to-point interface, ensuring data is sent in sync with a clock signal.



## Architecture

Based on a master-slave principle where one device controls the clock and data flow.

# SPI Interfaces

SPI communication is based on four dedicated single-ended lines:



**MOSI**

**Master Out,  
Slave In**

The line used by  
the Master to send  
data to the Slave.



**MISO**

**Master In, Slave  
Out**

The line used by  
the Slave to send  
data back to the  
Master.



**SCK**

**System Clock**

Synchronizes data  
transmission;  
generated by the  
Master.

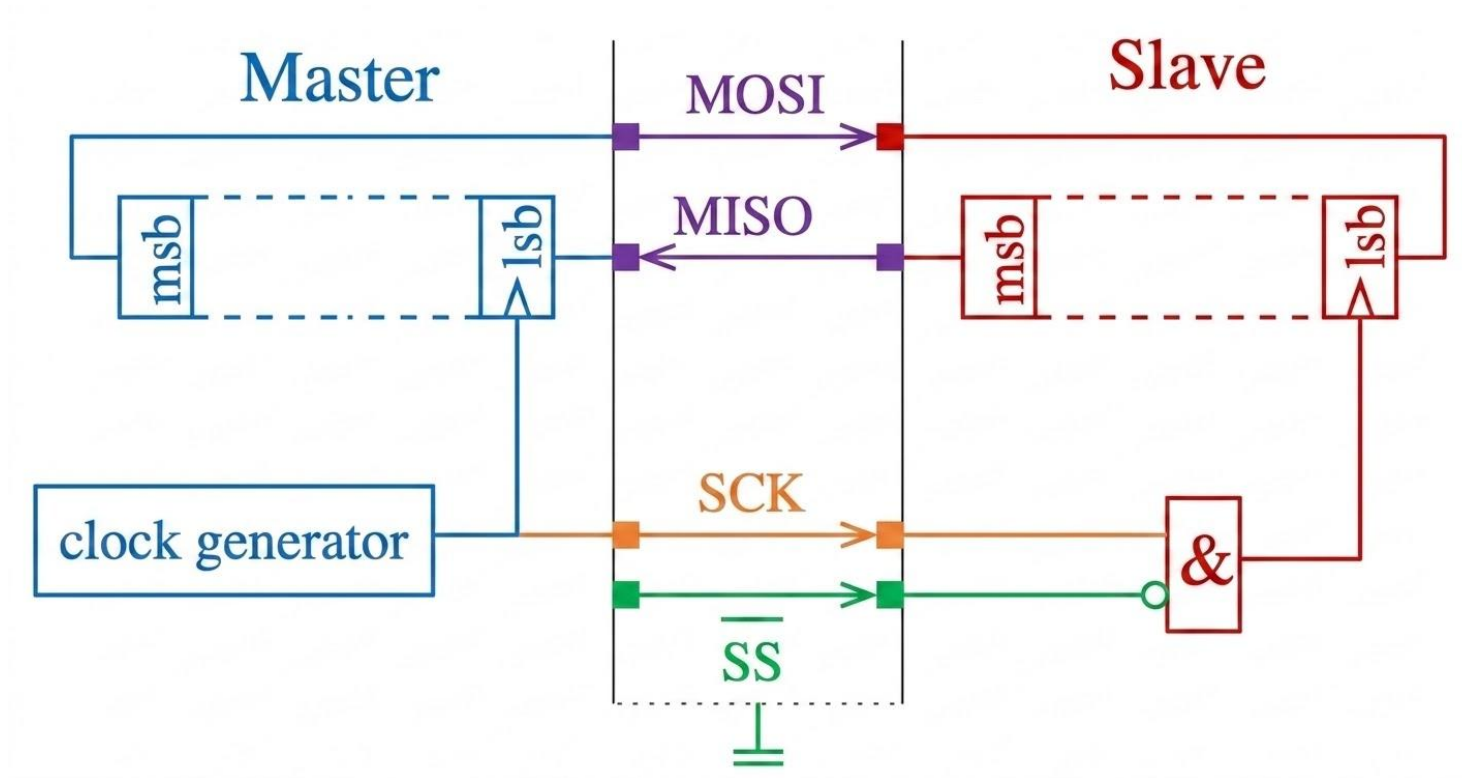


**SS**

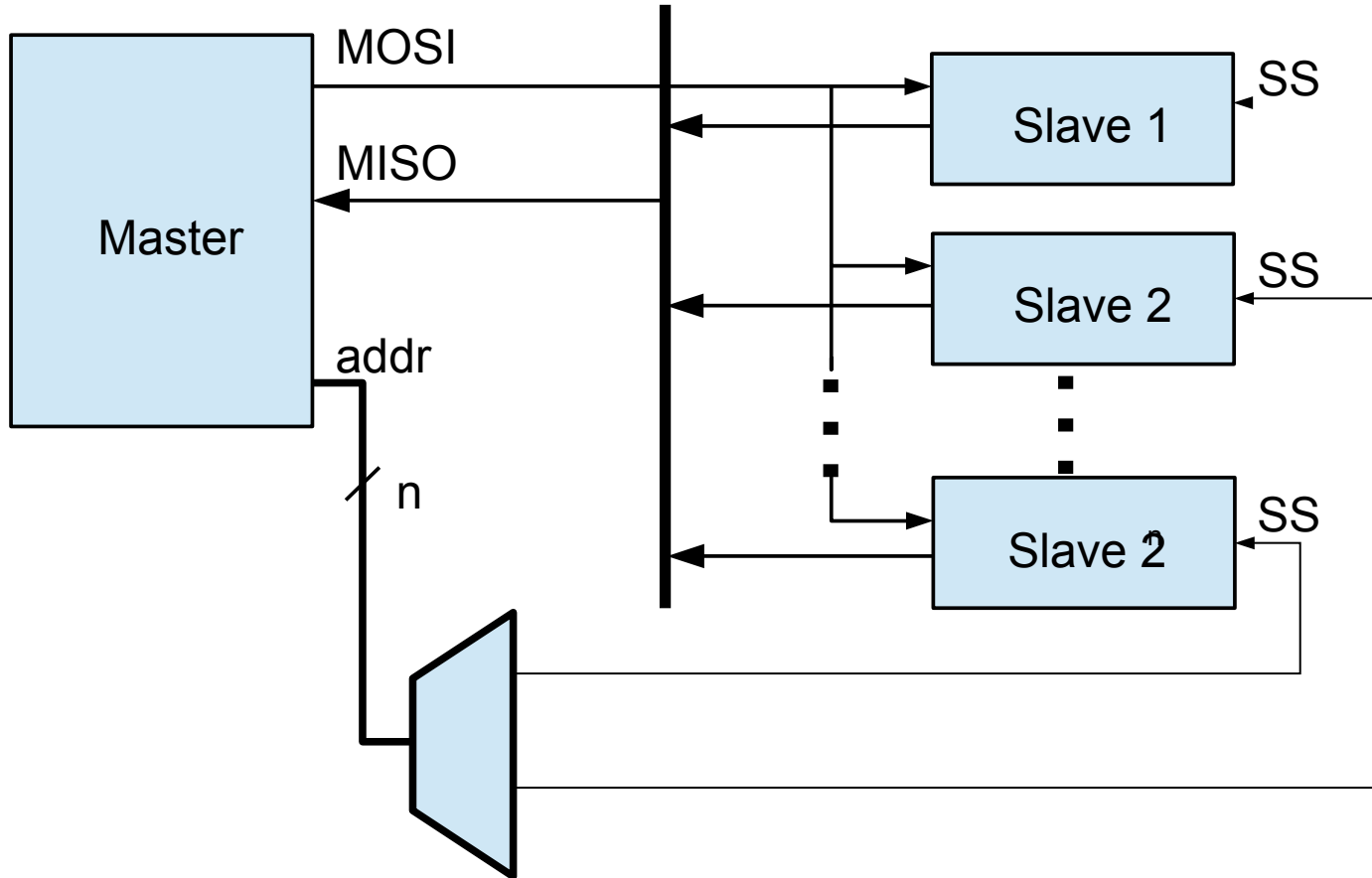
**Slave Select**

Enables a specific  
Slave for  
communication  
(often active low).

# SPI Functioning



# SPI Multiple Slaves





# I<sup>2</sup>C (Inter-Integrated Circuit)



## Inter-IC Bus

A multi-master, multi-slave, packet switched, single-ended, serial computer bus.



## Synchronous Interface

Uses a common clock signal (SCL) to synchronize data transfer between devices.

## Controller-Target Architecture

Based on the master-slave principle where the master initiates communication.

# I<sup>2</sup>C (Inter-Integrated Circuit)



## Physical Interface

Two single-ended wires:

- **SCL:** Serial Clock Line
- **SDA:** Serial Data Line



## Communication Type

Half-duplex communication

Devices can either transmit or receive data, but not both simultaneously.



## Origin & Design

Developed by Philips

Created in the early 1980s to allow easy communication between components on the same board.



## Common Usage

Short-distance communication

Widely used between one or more controllers and multiple peripheral devices.

# I<sup>2</sup>C Speed Modes



## Standard Mode

The baseline operational speed for I<sup>2</sup>C communication.

**Up to 100 kbit/s**



## Fast Mode

Increased throughput for more demanding data transfers.

**Up to 400 kbit/s**



## High-Speed Mode

Maximum performance for high-bandwidth applications.

**Up to 3.4 Mbit/s**



## Backward Compatibility & Mixing

Fast and high-speed devices can be seamlessly mixed with devices operating at lower speeds on the same bus.

# Data Transmission



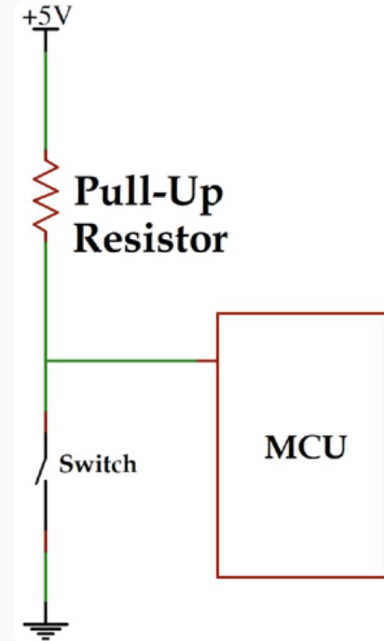
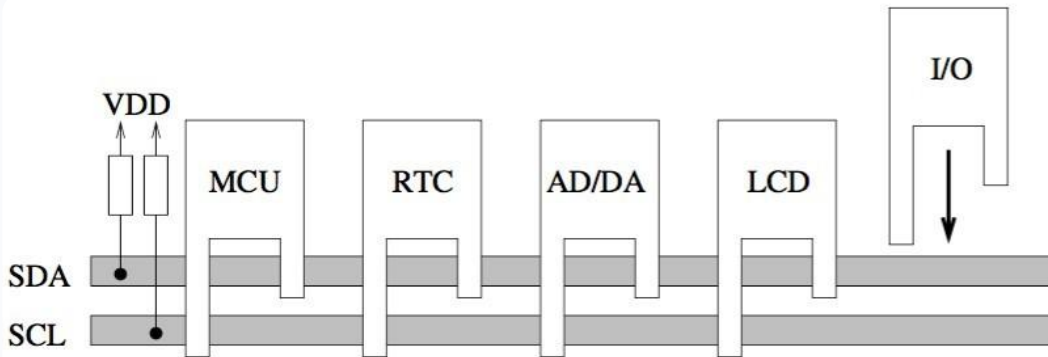
## Bus Mechanics

**Pull-up Resistors:** SDA and SCL wires are connected to VDD via pull-ups.

**Open-Drain:** The HIGH level is never actively driven by the sender.

**Passive Release:** Sender disconnects to let the line be pulled up externally.

**Active Drive:** Only the LOW level is actively driven to ground.



# Dominant and Recessive State



## Bus Logic Principle

If a device outputs 0 and thus drives the line low, it remains low even if one or more devices output 1 (i.e., disconnect their output).



## Collision Priority

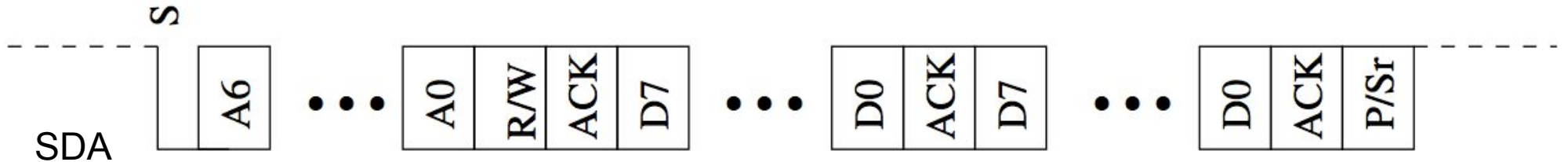
A 0 always wins over a 1



## State Definition

Low is **dominant**, high is **recessive**

# IIC Frame



- **S (Start):** The master device wakes up the bus to begin communication.
- **A6-A0 (Address):** The 7-bit identifier of the target slave device.
- **R/W (Read/Write):** Tells the target whether the master wants to write to it (0) or read from it (1).
- **ACK (Acknowledge):** A 1-bit signal from the receiver confirming it successfully got the previous 8 bits.
- **D7-D0 (Data):** The actual 8-bit message or payload being transferred.
- **P/Sr (Stop/Repeated Start):** Ends the current communication sequence (P) or immediately begins a new one without releasing the bus (Sr).

# Start & Stop Signals



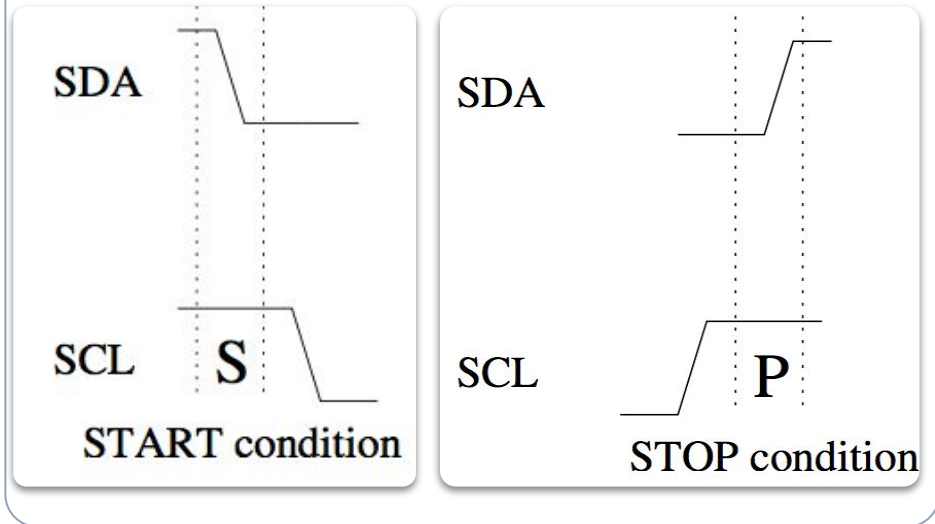
## Critical Timing Condition

START and STOP conditions are the **only** changes of SDA that can happen during a **high** state of SCL.



## Data Transmission

All other normal data transmission (including acknowledgments) change level during the **low** state of SCL.



# Data Transmission

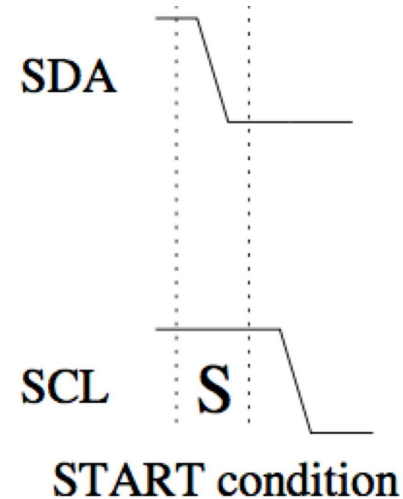
**Idle State:** Both SCL and SDA lines are high.



## Start Transmission

- The transmission is initiated by the **Master**.
- Master puts a **high clock signal** on SCL.
- Generates a **Start Condition (S)** on SDA.

## Timing Diagram: Start Condition





# I2C Address Phase

**Master Initiation:** The master places 7 address bits on the bus to target a specific slave device.

## Bit Timing

- **Change:** SDA level shifts during SCL **Low** phase.
- **Read:** SDA is sampled during SCL **High** phase.

## R/W Indicator

The 8th bit determines the access type:

- **Read:** Slave to Master
- **Write:** Master to Slave



# Data Transmission

**Address Acknowledgement:** The slave sends an acknowledge to indicate it recognized its address.

## Data Transfer

Either master or slave can transmit an arbitrary number of data bytes, depending on direction.

## Acknowledgement

Each byte transmitted is acknowledged by the receiver to ensure data integrity.



# Data Transmission (cont'd)

**End of Transmission Phase:** The master concludes the sequence using one of two methods.

## 1. STOP Condition (P)

The master releases the bus by generating a STOP signal.

- Indicates the bus is now **FREE**.
- Allows other masters to take control.

## 2. Repeated START (Sr)

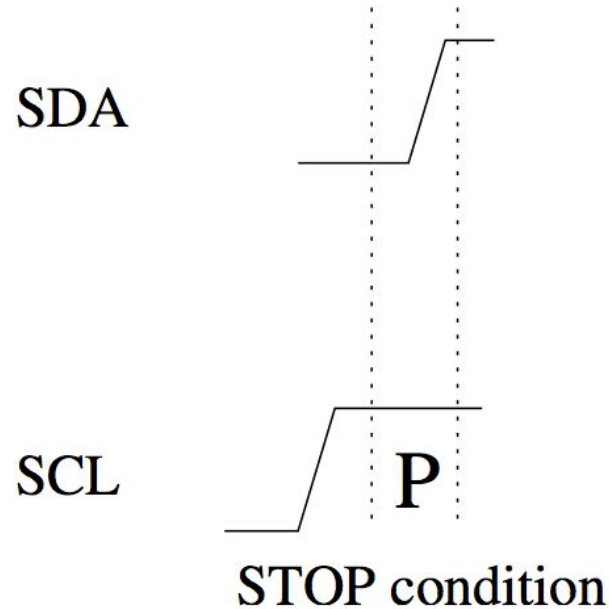
The master retains control of the bus for back-to-back transfers.

- Ends current transmission.
- Immediately **starts a new one** without losing priority.



# Stop

- STOP condition (P)
  - SDA goes from low to high while SCL is high



# I2C Acknowledgement (ACK)

**The Handshake:** The receiver confirms data reception every 8 bits.

## The ACK Signal

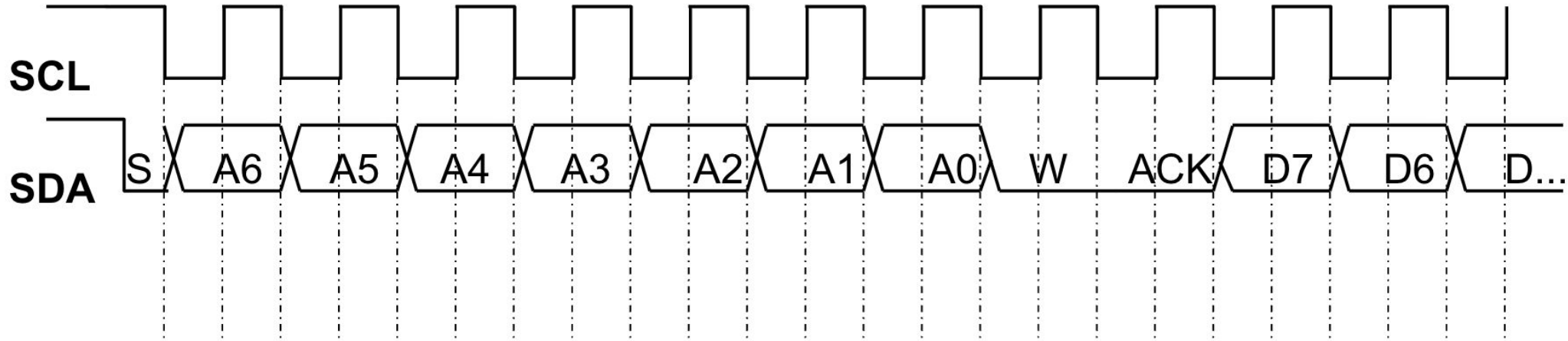
- Receiver pulls **SDA LOW** to acknowledge.
- Sent after every 8-bit data transfer.

## Purpose

- Indicates the receiver has **successfully** received the data.
- Provides flow control for error-free transmission.



# Example: 1-Byte Write Sequence



## Signal Logic

The diagram shows both high and low signals. Only one is transmitted based on the bit value (0 or 1).

## Timing Constraint

Values change during the **LOW clock** phase to be sampled during the **HIGH clock** phase.

*Exception: Start and Stop conditions.*

# Speed Control: Slave Clock Stretching

## The Constraint

Master transmits at a fixed speed matching slave specs; no initial negotiation exists.

## The Challenge

- Slaves may require additional processing time.
- A mechanism is needed to force the Master to wait.

## The Mechanism

- Exploits the **dominant/recessive** nature of the clock line.
- Slave pulls SCL LOW to pause transmission.

 *Note: This technique is commonly referred to as "Clock Stretching" in I2C communication protocol specifications.*

# Clock Stretching: Step-by-Step

## How the Slave Controls Speed

**🕒 Initiation:** Whenever the slave needs more processing time, it forcibly pulls the clock line (SCL) LOW.

**⏸ The Delay:** This action prolongs the LOW phase of the current clock tick, preventing the Master from starting the next cycle.

**▶ Resumption:** The Master remains in a wait state until the slave releases the SCL line, allowing it to go HIGH.

